

第 2 课 · 面试题库 (装工具 + Docker)

母题索引

母题	一句话	覆盖题号
N1 安装 ≠ 生效	程序在硬盘上 ≠ 服务在跑 ≠ 数据在采 ≠ 配置已读	13, 27, 29, 30, 40
N2 依赖与信任链	谁装的 (apt/dpkg) 、信不信 (GPG 签名) 、从哪来 (源)	2, 4, 5, 6, 10, 11, 18, 19, 28
N3 客户端 / 服务端分离	docker 只是客户端, 权限和配置的答案都在 daemon 与 socket 上	15, 24, 35, 36, 40
N4 隔离 vs 限制	namespace 管"看得见什么", cgroup 管"能用多少", 内核只有一份	14, 16, 17, 20, 26, 39
N5 权限的授予时机	组名单 (/etc/group) ≠ 进程凭证 (登录时固化)	21, 22, 23, 24
N6 最小权限原则	能小就不大: 非 root、只读根、按需加能力、只绑回环	21, 22, 25, 26, 33
N7 分层排障 (从里往外)	容器内 → 映射 → 宿主 → 防火墙, 一层层证伪	3, 9, 24, 31, 32, 33, 34, 38, 41
N8 数据与状态的归属	容器可写层会消失, 数据必须在容器之外	12, 19, 29, 37, 38
N9 可审计 / 可复现 / 可回滚	用官方源、锁版本、留痕迹 —— 运维和"手快"的区别	1, 7, 8, 18, 38

问题清单

A 组 · 包管理 (任务是装工具)

- ☐ 1. 【★★】 apt 和 apt-get 有什么区别? 写脚本时该用哪个?
- ☐ 2. 【★★】 apt 和 dpkg 是什么关系?
- ☐ 3. 【★★】 怎么查一个命令或文件属于哪个软件包?
- ☐ 4. 【★★】 怎么查看一个软件包安装了哪些文件?
- ☐ 7. 【★★】 apt update 和 apt upgrade 的区别? full-upgrade 呢?
- ☐ 10. 【★★】 apt 报 NO_PUBKEY 或签名验证失败怎么办? signed-by 是干什么的?
- ☐ 11. 【★★】 怎么查看系统里已安装的包? 怎么彻底卸载一个包 (连配置一起删)?
- ☐ 5. 【🔧】 apt 的软件源配置在哪些文件里? Ubuntu 24.04 之后有什么变化?
- ☐ 6. 【🔧】 换软件源的完整流程是什么? 换完必须执行什么?
- ☐ 8. 【🔧】 怎么把某个包锁定在某个版本, 不让它被升级?
- ☐ 9. 【🔧】 报 Could not get lock /var/lib/dpkg/lock-frontent 怎么处理? 为什么不能直接删锁文件?
- ☐ 13. 【🔧】 sysstat 装好了, 但 sar 查历史一直是空的, 为什么?
- ☐ 12. 【☆】 apt autoremove 和 apt clean 的作用与风险?

B 组 · Docker 原理 (任务是装 Docker)

- ☐ 14. 【★★★★】 Docker 和虚拟机的本质区别是什么?
- ☐ 15. 【★★★★】 Docker 的架构链路是怎样的? docker 命令到底把请求发给了谁?
- ☐ 16. 【★★★★】 namespace 和 cgroup 各自负责什么?
- ☐ 19. 【★★★★】 镜像分层和 overlay2 是什么? 构建缓存有什么用?
- ☐ 17. 【★★】 容器为什么"启动快"?
- ☐ 18. 【★★】 你怎么安装 Docker? 为什么推荐官方 apt 源而不是 curl | sh?
- ☐ 20. 【★★】 容器里的进程, 在宿主机上能看到吗?

C 组 · 权限与安全 (本课重点: docker 组)

- ☐ 21. 【★★★★】 为什么说 docker 组等价于 root? 怎么证明?
- ☐ 22. 【★★★★】 不用 docker 组, 还有哪些更安全的方式让普通用户用 Docker?
- ☐ 23. 【★★★★】 usermod -aG docker 之后, 为什么必须重新登录?
- ☐ 24. 【★★★★】 docker ps 报 permission denied, 怎么排查?
- ☐ 25. 【★★★★】 --privileged 有什么风险? 更细粒度的替代方案是什么?
- ☐ 26. 【★★】 容器里的 root 和宿主机的 root 是同一个吗?

D 组 · 配置、日志与排障 (任务 D/E)

- ☐ 27. 【★★★★】 daemon.json 改完怎么才能生效? 会影响正在运行的容器吗? live-restore 是什么?
- ☐ 29. 【★★★★】 为什么生产环境必须配日志轮转? 怎么给已经存在的容器加上限制?
- ☐ 31. 【★★★★】 Docker daemon 起不来, 怎么排查?
- ☐ 32. 【★★★★】 docker run 成功, 但外部 curl 不通, 怎么排?
- ☐ 33. 【★★★★】 -p 8080:80 是什么意思? -p 127.0.0.1:8080:80 有什么不同? -P 呢?
- ☐ 36. 【★★★★】 docker exec 和 docker attach 的区别?
- ☐ 37. 【★★★★】 容器里的数据会丢吗? volume 和 bind mount 怎么选?
- ☐ 38. 【★★★★】 磁盘被 Docker 吃满, 怎么清理? 清理顺序和风险是什么?
- ☐ 30. 【★★】 json-file / local / journald 日志驱动怎么选?
- ☐ 35. 【★★】 docker run 的 -d / -it / --rm 分别是什么?
- ☐ 39. 【★★】 容器里时间不对 (差 8 小时), 怎么处理?
- ☐ 40. 【★★】 docker run --rm hello-world 到底验证了什么?
- ☐ 28. 【🔧】 registry-mirrors 的作用范围和限制?

- 34. 【🔧】起容器时报端口已被占用，怎么查是谁占的？
- 41. 【★】为什么运维验证服务要用 `curl -I` 而不是打开浏览器？

参考答案

【★★】1 • apt 和 apt-get 的区别？脚本里用哪个？

- **结论：**apt 是给人用的（有进度条、颜色、更好看的输出）；apt-get 是给脚本用的（输出稳定、格式不变、无交互警告）。
- **展开：**两者底层是同一套库，功能几乎等价。apt 出现就是为了终结 apt-get / apt-cache / apt-config 三件套的混乱。
- **加分点：**写自动化脚本、CI/CD 时用 apt-get（因为 apt 会打一句 "WARNING: apt does not have a stable CLI interface"，而且它输出会随版本变）；日常手动操作用 apt。

【★★】2 • apt 和 dpkg 是什么关系？

- **一句话：**dpkg 是底层（操作单个 .deb：解包、安装、写入数据库），apt 是上层（解决"从哪下、依赖谁、先装谁"）。
- **展开：**dpkg -i xxx.deb 不做依赖解析，缺依赖就报错停在那里；apt install 会自动把依赖一起装好。
- **加分点：**apt 本身不干活，它调用 dpkg；所以 apt 报依赖错误时，dpkg --configure -a 是常用的修复动作。

【★★】3 • 怎么查一个命令属于哪个包？

- **已安装的：**dpkg -S \$(which nginx) 或 dpkg -S /usr/sbin/nginx。
- **还没装的：**apt-file search nginx（需先 apt install apt-file && apt-file update）。
- **加分点：**"生产机上只有二进制、不知道它是从哪来的"是常见场景 —— which + dpkg -S 两跳就能定位，比乱猜包名快得多。

【★★】4 • 怎么查看一个包装了哪些文件？

- dpkg -L nginx 列出该包安装的所有文件。
- 反向的还有 dpkg -l（看包状态）、dpkg -s nginx（看包的元信息和依赖）。
- **加分点：**查"配置文件在哪、有没有残留"用得上 —— 卸载后 dpkg -L 还能列出该包曾管理的路径。

【🔧】5 • apt 源配置在哪？24.04 之后有什么变化？

- **位置：**/etc/apt/sources.list 和 /etc/apt/sources.list.d/*.list（每行一个源）。
- **变化：**Ubuntu 24.04 起默认改成 deb822 格式，文件是 /etc/apt/sources.list.d/ubuntu.sources，内容形如 Types: deb / URIs: / Suites: / Components:，而且 /etc/apt/sources.list 可能已经不存在或只剩注释。
- **加分点：**这解释了"为什么照旧教程改 sources.list 没反应" —— 排故障时先 ls 看清楚用的是哪种格式，再动手。

【🔧】6 • 换源的完整流程？换完必须做什么？

- **流程：**① 备份原文件（cp -a）；② 把 archive.ubuntu.com / security.ubuntu.com 替换成国内镜像（如 mirrors.aliyun.com、mirrors.tuna.tsinghua.edu.cn）；③ **必须** sudo apt update 刷新索引。
- **为什么必须 update：**apt 用的是本地缓存的索引（/var/lib/apt/lists/），换了源不刷新，装的还是旧源的元数据；更严重的是"源和索引对不上"会报 hash 校验失败。
- **加分点：**"换源 → apt update → 验证能装"三步才算完成；只改文件不 update 是新手最常见的"换了源没用"。

【★★】7 • apt update / upgrade / full-upgrade

- apt update：只刷新软件包索引（"有哪些新版本可用"），不装任何东西。
- apt upgrade：升级所有包，**绝不删除已有包**；需要删包才能解决的依赖变化会被跳过。
- apt full-upgrade（旧名 dist-upgrade）：允许为满足依赖而增删包，能完成大头升级 —— 能力更大，风险也更大。
- **加分点：**生产上"能不能随便 upgrade"要问清楚；**内核升级后需要重启才生效**（needrestart / ls /var/run/reboot-required），这是"打补丁没生效"的经典坑。

【🔧】8 • 怎么锁定包的版本？

- sudo apt-mark hold nginx 锁定；apt-mark unhold nginx 解锁；apt-mark showhold 查看。
- 也可以写 /etc/apt/preferences.d/ 做更细的版本优先级（Pin）。
- **加分点：**面试话术 —— "生产上我倾向锁住关键组件版本，用镜像或内部源统一管理，避免不同机器装出不同版本，这就是**不可变基础设施**的思路"。这一句能让你从"会敲命令"抬到"懂运维"。

【🔧】9 • apt 报 dpkg lock 怎么办？为什么不能删锁文件？

- **原因：**已经有另一个 apt / dpkg 进程在跑（常见是 unattended-upgrades 在后台自动升级）。
- **正确做法：**先查 ps aux | grep -E "apt|dpkg"；等它结束，或停掉自动升级（systemctl stop unattended-upgrades）等它收尾。
- **为什么不能删锁：**那个锁保护的是 dpkg 的数据库状态。硬删锁会让两个进程同时改数据库，可能把包管理系统写坏，最后要靠手动 dpkg --configure -a 甚至重装系统级依赖。
- **加分点：**正确姿势是"找到占用者，让它体面结束"，不是"删掉障碍物"。这句话是运维价值观的体现。

【★★】10 • NO_PUBKEY / 签名失败怎么办？signed-by 是什么？

- **为什么验签：**apt 从网络下载包，签名用来确认"这个包确实来自官方、没被中间人换过"。
- **signed-by：**在源的配置里指定"只信这把公钥"，把信任范围收缩到单个源，比已经废弃的全局 apt-key add 安全得多。
- **修复：**把对应仓库的公钥下载并转成 keyring 文件：

```
curl -fsSL <仓库公钥URL> | sudo gpg --dearmor -o /etc/apt/keyrings/<名字>.gpg, 然后在源里写 signed-by=/etc/apt/keyrings/<名字>.gpg。
```

- **加分点：**这也是第 2 课装 Docker 的第 ① 步在干什么 —— **别把 apt-key 当成万能药**（它在 Ubuntu 22.04 起已弃用）。

【★★】11 • 查看已安装的包 / 彻底卸载

- 查看: `dpkg -l` (状态 + 版本 + 描述)、`apt list --installed`。
- 彻底卸载: `sudo apt purge <包>` (连配置文件一起删) → `sudo apt autoremove` (清理自动装的依赖)。
- 区别: `remove` 保留配置, `purge` 删配置。
- 加分点: 重装服务前用 `purge`, 否则旧配置会“复活”并覆盖你的预期 —— 这是“我明明改了配置怎么又变回去了”的常见原因。

【☆】12 • apt autoremove / clean 的作用与风险

- `autoremove`: 删掉“当初作为依赖自动装上、现在没人需要”的包。风险: 依赖判断偶有误伤, 生产上先 `--dry-run` 看一眼再执行。
- `clean`: 清空 `/var/cache/apt/archives/` 里下载的 `.deb` 缓存, 只省磁盘、不动已装的包, 相对安全。
- 加分点: `/var/cache/apt` 也是磁盘占用的排查点之一; 但清理缓存不是排障手段, 它只是省空间。

【🔧】13 • sysstat 装了, sar 却查不到数据?

- 原因: `sysstat` 的数据采集开关默认是关的 (`/etc/default/sysstat` 里 `ENABLED="false"`), 而且采集靠 `sysstat` 这个 `systemd` 服务 + `cron/timer` 定时跑。
- 修: 把 `ENABLED` 改成 `true`, 然后 `sudo systemctl enable --now sysstat`; 数据落在 `/var/log/sysstat/` (Debian/Ubuntu) 或 `/var/log/sa/` (RHEL 系)。
- 面试题: 这题的真正考点是 “安装 ≠ 生效” —— 程序在硬盘上 ≠ 服务在跑 ≠ 数据在采。这是运维的第一性思维, 几乎所有服务 (`nginx/redis/mysql`) 都适用。

【★★★】14 • Docker 和虚拟机的本质区别?

- 一句话: 容器是进程级隔离 (共享宿主内核, 用 `namespace + cgroup` 圈出一块天地); 虚拟机是硬件级虚拟化 (Hypervisor 造出一台假电脑, 装一个完整的内核和系统)。
- 对比:

维度	容器	虚拟机
内核	共享宿主机内核	每个 VM 一个独立内核
启动	秒级 (本质是起一个进程)	分钟级 (要走完整启动流程)
体积	镜像 MB 级	系统 GB 级
隔离强度	较弱 (内核是共用面)	强 (硬件层隔离)
密度	一台机器几百个很常见	一台机器几十个就很重

- 加分点: 结论不是“Docker 取代 VM”, 而是两者分工 —— 强隔离/异构系统/内核定制用 VM; 快速交付、弹性伸缩、无状态服务用容器。云上现实是 VM 里跑容器 (ECS + Docker/K8s), 这句话能直接对面试官的心智。

【★★★】15 • Docker 的架构链路? 请求发给了谁?

- 链路: `docker` (CLI 客户端) → `/var/run/docker.sock` (UNIX socket) → `dockerd` (守护进程, 管镜像/网络/卷) → `containerd` (容器生命周期) → `runc` (真正调用内核创建容器, 创建完就退出) → `shim` 进程看住容器。
- 关键认知: `docker` 命令只是个客户端, 它不干活 —— 这一点想通了, 很多问题自然有答案:
- 为什么“docker 命令有了但 daemon 没起”会报 `Cannot connect to the Docker daemon`;
- 为什么免 `sudo` 的关键是 `socket` 的权限 (属主 `root`、组 `docker`、`0660`);
- 为什么“容器里不能改宿主机内核” (`runc` 只是调内核 API)。
- 加分点: 能远程管 Docker (`DOCKER_HOST=tcp://...`) 就是这条链路的延伸, 也正是“别把 `docker.sock` 暴露给不可信进程”的原因 —— 拿到它就等于拿到宿主 `root`。

【★★★】16 • namespace 和 cgroup 各负责什么?

- `namespace` (看见什么): `PID`、`Mount`、`Network`、`UTS` (主机名)、`IPC`、`User`、`Cgroup`、`Time` —— 每一个都让容器“以为是独一份”。隔离。
- `cgroup` (能用多少): `CPU`、内存、`IO`、进程数上限 (`pids`)。限制。第 3 周 `systemd-run --scope -p MemoryMax=4G` 用的就是它。
- 加分点: 现代内核默认 `cgroup v2` (统一层级) —— 这是“老教程的 `cgroup` 路径和现在不一样”的原因 (`/sys/fs/cgroup` 结构变了)。别把 `cgroup v2` 当成新版本特性去找故障原因。

【★★】17 • 容器为什么启动快?

- 不引导内核、不走完整 `init` 流程, 直接 `exec` 一个进程就算启动完成。
- 镜像分层 + 本地已有缓存, 不需要“装机”。
- 但要记住前提: 镜像没在本地时第一次仍然要拉几百 MB —— 所以“容器秒起”在生产上依赖镜像预热 (镜像仓库就近、节点提前 `pull`、`imagePullPolicy` 策略)。
- 加分点: 能说出“慢的部分在拉镜像、不在启动”的人, 通常真的调过 K8s。

【★★】18 • 怎么装 Docker? 为什么不用 curl | sh?

1. 方式: 加官方 GPG 公钥 (`/etc/apt/keyrings/docker.gpg`) → 加官方 apt 源 → `apt install docker-ce ...` → `usermod -aG docker` → 重新登录。
2. 为什么不用 `curl -fsSL https://get.docker.com | sudo sh`:
3. 不可审计: 脚本内容随时会变, 你无法知道它到底改了什么;
4. 不可复现: 今天装的结果和下周装的不一样;
5. 没有签名校验: 拿到的是脚本而不是签名的包, 中间人可替换;
6. 不好回滚: 装了什么、怎么卸, 说不清。

7. 加分点：企业里还会更进一步 —— **内网私有源 + 固定版本锁**（“同一套镜像在测试/生产装的 Docker 版本完全一致”），这是“规范化”和“凭手感”的分水岭。

【★★★】19 • 镜像分层和 overlay2? 构建缓存有什么用?

- **分层**：Dockerfile 里**每条指令生成一层只读层**，镜像 = 若干只读层叠加；容器启动时在最上面加一层**可写层（Copy-on-Write）**。多个镜像的相同层**共享一份磁盘**。
- **overlay2**：现代 Docker 的默认存储驱动，用联合挂载把多层“合”成一个文件系统视图。
- **构建缓存的价值**：指令没变 → 直接复用那层，秒级完成；所以 Dockerfile 有经典写法 —— **把变化频率低的放前面**（先 COPY package.json + RUN npm install，最后才 COPY .），这样改代码时不用重装依赖。
- **加分点**：容器**可写层随容器删除而消失** —— 这就是“容器里存数据会丢”的根本原因，也是必须用 volume 的数学解释。

【★★】20 • 容器里的进程，宿主上能看到吗？

- **能看到**。容器进程在宿主上就是**普通进程**（ps -ef 可见），或 docker top <容器> 直接看。
- **区别只在“视角”**：PID namespace 让容器内看到的 PID 从 1 开始，宿主上看到的是普通 PID。
- **加分点**：**全机器只有一个内核、一张进程表**。所以：
- 容器里 kill -9 1 会把容器搞停（PID 1 是容器主进程）；
- 宿主上“杀容器进程”等于杀容器；
- 排查容器 CPU/内存异常，可以在**宿主用 top/pidstat 看真实数字** —— 这是实战里最快的路径。

【★★★】21 • 为什么说 docker 组等价于 root? 怎么证明？

- **原因**：能起容器的人，可以把**宿主根目录挂进容器**或开 --privileged，进而在容器里以 root 身份改宿主文件 —— 等于直接拿到宿主 root。
- **证明（面试可直接口述）**：
docker run -v /:/host -it alpine chroot /host —— 一行就把宿主根目录当自己的根用；或 docker run --privileged --pid=host -it alpine nsenter -t 1 -m -u -i -n -p sh 直接钻进宿主 PID 1 的所有命名空间。
- **加分点**：所以“给某人加 docker 组”在规范的公司里**要走审批**，属于特权账号管理；把它说成“图个方便”是明显的安全意识缺失。

【★★★】22 • 不用 docker 组，还有哪些更安全的方式？

- **rootless Docker**：dockerd-rootless-setup.sh install —— 用 user namespace 把 root 映射到普通用户，**不需要 docker 组**；代价是部分功能受限（绑定 <1024 端口要额外配置、网络与存储驱动受限、性能略降）。
- **Socket 代理**：只暴露需要的 API（如只允许 GET 容器列表），不把完整 socket 交出去。
- **Podman**：无守护进程、原生 rootless，企业内越来越常见。
- **sudo 白名单**：只放行极少数固定命令（能用但不优雅，权限仍然很宽）。
- **真正的答案**：**生产环境不该让人手工 docker run** —— 应该用 K8s + RBAC、CI/CD 流水线，把“谁能操作什么”变成权限系统管的事。
- **加分点**：把“权限最小化 + 审计”这两个词讲出来，就已经站在运维岗位的语境内了。

【★★★】23 • usermod -aG docker 后为什么必须重新登录？

- **原理**：进程属于哪些组，是**登录那一刻**由 PAM 调 setgroups() 写进**进程凭证（内核 task_struct 里的组列表）**的。usermod 只改了 /etc/group 这个“名单文件”，**不会去更新已经存在的进程**。
- **现象**：getent group docker 里有你，id -nG 里没你 —— 同一个事实，两条命令答案相反。
- **修**：完全退出登录（注销 / 断开 ssh 重连）或 newgrp docker（临时、只对当前 shell 生效）。
- **加分点**：这条规律对所有“改用户/组关系”的操作都成立（usermod、gpsswd、sudoers 改动）—— 第 2 周改 sudoers 那天的“保持一个不关的 root shell”就是这个原理的反向应用。

【★★★】24 • docker ps 报 permission denied 怎么排查？

- **四步**：① id -nG 看有没有 docker 组；② 有但没生效 → **重新登录**（见 23）；③ 看 socket 权限 ls -l /var/run/docker.sock（应为 srw-rw---- root docker）；④ sudo docker ps 能不能成功 —— **能成功就说明 daemon 是好的，纯粹是权限问题**。
- **不要做**：chmod 777 /var/run/docker.sock —— 那等于把 root 权限发给全机器所有人，是严重的安全反模式。
- **加分点**：这题考的是**分类能力**：先分清“服务问题”和“权限问题”，再往下走。先分类、后动手，是排障效率的分水岭。

【★★★】25 • --privileged 的风险？更细粒度的替代？

- **风险**：--privileged 会**关闭几乎全部隔离与能力限制**，并让容器可以看到所有设备 —— 等于把宿主 root 交出去。
- **替代（按推荐度）**：
- 默认丢弃所有能力，只按需加：--cap-drop=ALL --cap-add=NET_BIND_SERVICE
- 以非 root 用户运行：--user 1000:1000
- 只读根文件系统：--read-only（需要写的目录用 tmpfs 或 volume）
- 禁止提权：--security-opt=no-new-privileges
- 精确授权设备：用 --device 代替把整个宿主交出去
- 保留默认 seccomp / AppArmor 配置，不要 --security-opt seccomp=unconfined
- **加分点**：一句话 —— “我不用 --privileged，我用最小能力集 + 非 root + 只读根文件系统凑出来。”

【★★】26 • 容器的 root 和宿主的 root 是同一个吗？

- **默认是同一个**：不开 user namespace 时，容器里的 uid 0 就是宿主上的 uid 0 —— 所以一旦逃逸，直接就是宿主 root。
- **开了 userns-remap / rootless 后不同**：uid 会被映射到宿主的高位普通 uid（如 100000+），即使容器里是 root，落到宿主也只是普通用户，风险显著降低。

- **加分点：**这是“容器隔离不等于安全边界”的技术根据，也是为什么 K8s 里流行 `runAsNonRoot + readOnlyRootFilesystem + Pod Security Standards` 的原因。

【★★★】27 • daemon.json 改完怎么生效？影响运行中的容器吗？

- **生效方式：**`sudo systemctl restart docker`。`daemon.json` 没有热加载（`systemctl reload docker` 不适用）。
- `systemctl daemon-reload` 是另一件事：它刷新的是 `systemd` 的 **unit 文件**，跟 `daemon.json` 无关——但两条一起敲无害，社区命令里常见。
- **对运行中容器的影响：**默认 `restart docker` 会停掉正在运行的容器；配上 `"live-restore": true` 后，`daemon` 重启/升级时容器继续运行（注意：Swarm/K8s 场景一般不开，因为编排层自己管）。
- **加分点：**改错 `daemon.json`（多一个逗号）→ `daemon` 直接起不来 → 这时 `systemctl status` 只说 failed，真正的原因在 `sudo journalctl -xeu docker | tail -20`。这个组合是本课最重要的排障反射。

【🔧】28 • registry-mirrors 的作用范围和限制？

- **作用范围：**只对 **Docker Hub (docker.io)** 的 pull 生效。拉 `quay.io`、`ghcr.io`、私有仓库不走它。
- **不代理 push：**推镜像慢是另一回事，配了加速器也不会变快。
- **生效时机：**`daemon` 启动时读入，所以改完必须 `restart docker`；验证用 `docker info | grep -A3 "Registry Mirrors"`。
- **加分点：**企业里的正规做法是自建 **Harbor + 代理缓存 (pull-through cache)**，一来稳、二来有审计、三来能离线分发——比堆公网镜像源专业得多。

【★★★】29 • 为什么必须配日志轮转？怎么给已有的容器加限制？

- **不配的后果：**默认 `json-file` 没有大小上限，容器往 `stdout` 一直写，宿主磁盘就被吃满——这是“服务跑得好好的突然全挂”的经典事故。
- **只对新建容器生效：**`log-opts` 写在 `daemon.json` 里，已存在的容器不会变（因为配置在容器创建时就固化了）。
- **怎么处理已有容器：**① 重建容器（推荐，顺带规范配置）；② 应急先截断日志文件：`truncate -s 0 $(docker inspect --format='{{.LogPath}}' <容器>)`。
- **加分点：**更进一步是**日志集中收集**（Loki / ELK / 云日志服务），本地只留 3 天——“本地日志只做排障，长期留痕交给日志平台”，这句话很加分。

【★★】30 • json-file / local / journald 怎么选？

- `json-file`：**默认**，人可读、`docker logs` 支持、**不限制大小**（必须配 `log-opts`）。
- `local`：二进制格式，**更省空间、写入更快，默认就有大小上限**——生产上更推荐。
- `journald`：日志交给 `systemd-journald`，用 `journalctl` 统一查（和大盘其他服务一起看），但要单独配 `journald.conf` 的轮转。
- `syslog / fluentd / awslogs`：发到集中收集端，适合多机汇聚。
- **加分点：**很多驱动**不支持 docker logs**（如 `syslog`），排障前先确认日志去了哪——面试里说出“日志驱动是每个容器可单独覆盖的”，说明你真配过。

【★★★】31 • Docker daemon 起不来，怎么排查？

- **固定顺序：**`systemctl status docker`（看状态）→ `sudo journalctl -xeu docker | tail -50`（看真正的原因）。
- **高频原因：**① `daemon.json` JSON 语法错；② 磁盘满（`df -h`）；③ `cgroup` 驱动与 `kubelet` 不一致（`systemd` vs `cgroupfs`）；④ `overlay2` 不可用 / 内核模块缺失；⑤ `iptables` 后端冲突；⑥ `containerd` 没起来；⑦ 端口或 `socket` 被别的进程占。
- **加分点：**实在看不出就**前台跑一次** `sudo dockerd --debug`，日志会直接打在屏幕上，比翻 `journal` 更快定位。

【★★★】32 • docker run 成功，但外部 curl 不通，怎么排？

1. **分层定位（从里往外）：**
2. **容器里：**`docker exec -it <容器> ss -lntp`——进程在监听吗？是 **0.0.0.0** 还是 **127.0.0.1**？（只监听 `127.0.0.1` 就永远外部进不来）
3. **映射对不对：**`docker port <容器>`、`docker ps` 看 `0.0.0.0:8080->80/tcp`。
4. **宿主机：**`curl -I 127.0.0.1:8080`——通说明映射没问题，是“再往外”的问题。
5. **防火墙/安全组：**`sudo iptables -L -n`、云上还要看安全组。
6. **必答的坑：**`Docker` 直接写 `iptables` 的 `nat/forward` 链，会绕过 `ufw` 的 `INPUT` 规则——你以为 `ufw` 拦住了，其实容器端口已经暴露，反之亦然；要在 `DOCKER-USER` 链里管。
7. **加分点：**先给结论“我先确认监听地址，再确认映射，最后才怀疑防火墙”——顺序对了，面试官就知道你真排过。

【★★★】33 • -p 8080:80 / -p 127.0.0.1:8080:80 / -P

- `-p 8080:80`：**宿主 8080 → 容器 80**（默认 TCP；`-p 8080:80/udp` 指 UDP）。
- `-p 127.0.0.1:8080:80`：只在**宿主回环**上暴露，**只有本机能访问**——更安全，生产上常配合前置 `nginx` 反代使用。
- `-P`：把镜像里 `EXPOSE` 的端口**随机映射**到宿主高端口，用 `docker port` 查。
- **底层实现：**`iptables` 的 `DNAT` 规则 + `docker-proxy`（`userland proxy`）兜底，所以“端口通了”其实是被 `iptables` 转发的。
- **加分点：**能说出 `-p 127.0.0.1:...` 这个用法，说明你有**最小暴露面**的意识。

【🔧】34 • 端口被占用怎么办？

- `sudo ss -lntp | grep 8080`（推荐，`ss` 属于 `iproute2`）或 `sudo lsof -i :8080`。
- 也要看是不是**别的容器**占着：`docker ps --format '{{.Names}} {{.Ports}}'`。
- **加分点：**容器删了但端口还占着 → 常见是 `docker-proxy` 残留或宿主进程占着；这时查 `ss` 输出里的 `pid/进程名` 一栏，别瞎重启机器。

【★★】35 • -d / -it / --rm 分别是什么？

- `-d`：后台运行（detach），命令行立刻返回容器 ID。
- `-it`：`-i` 保持标准输入 + `-t` 分配伪终端 —— 用来“进去敲命令”，缺一个就进不了正常交互。
- `--rm`：容器退出即自动删除，适合一次性任务（如 `hello-world`、跑个测试）。
- 加分点：不写 `--rm` 又频繁起临时容器，会积累一堆 `Exited` 容器占磁盘 —— 这是我每周 `docker system df` 的原因。

【★★★】36 • docker exec 和 docker attach 的区别？

- `exec`：在运行中的容器里新起一个进程（如 `exec -it c bash`），退出不影响主进程 —— 日常排障首选。
- `attach`：接到容器 **PID 1** 的标准输入输出上。风险：`Ctrl+C` 可能把主进程一起干掉；多个终端同时 `attach` 会互相看到对方的输出。
- 加分点：镜像太精简没有 shell（`scratch/distroless`）时 `exec` 进不去，可以用 `docker run --pid=container:<容器> --net=container:<容器> --ipc=container:<容器> -it alpine nsenter ...` 或 `docker debug` 借工具 —— 这说明你区分“容器没 shell”和“容器坏了”。

【★★★】37 • 容器数据会丢吗？volume 和 bind mount 怎么选？

- 会丢：容器可写层随容器删除而消失（`docker rm` 即没），镜像更新重建容器更会没。
- **volume**：由 Docker 管理，落在 `/var/lib/docker/volumes/`，跨平台、易备份、权限由 Docker 处理 —— **生产首选**。
- **bind mount**：直接挂宿主目录（`-v /data/app:/app`），直观、方便和宿主/其他容器共享配置，但强依赖宿主路径与权限，可移植性差。
- 加分点：真正的答案是“数据在容器之外” —— 数据库这类有状态服务用 volume + 定期备份/快照；无状态服务直接不落盘。“无状态化”是容器化的核心设计原则，说出这句就到位了。

【★★★】38 • 磁盘被 Docker 吃满，怎么清理？

1. 先看：`docker system df`（分类看：镜像 / 容器 / 卷 / 构建缓存）。
2. 清理顺序（按安全度从高到低）：
3. `docker builder prune` —— 构建缓存，通常是最大头，最该清
4. `docker image prune` —— 悬空镜像（`<none>`）
5. `docker container prune` —— 已停止的容器
6. `docker system prune` —— 上面几类打包清
7. `docker image prune -a` —— 未使用镜像（谨慎：镜像源不稳时要重新拉几百 MB）
8. `docker volume prune` —— 未使用的卷，可能直接删掉数据，最危险，先想清楚
9. 加分点：能说出“我按构建缓存 → 悬空镜像 → 未使用镜像 → 卷的顺序清，越往后越谨慎”，比背一条 `prune -a` 专业得多。

【★★】39 • 容器里时间不对（差 8 小时）怎么处理？

- 先分清两件事：
- 时钟（现在几点）：容器与宿主共享内核时钟，跑得一样 —— 有偏差说明是宿主/虚拟机的时钟漂移（虚拟机挂起恢复后常见，需要 `timesync / chrony / open-vm-tools`）。
- 时区（怎么显示）：容器默认继承镜像里的 `/etc/localtime`，常是 UTC → 于是“差 8 小时”。
- 修时区：`docker run -e TZ=Asia/Shanghai ...` 或挂载 `-v /etc/localtime:/etc/localtime:ro`。
- 加分点：生产规范是日志统一用 UTC，展示层再转本地时区 —— 这样多地域服务排障时时间轴才对得上。

【★★】40 • docker run --rm hello-world 验证了什么？

- 它验证的是一条完整链路：① CLI 能连上 daemon（socket 权限/组是否生效）；② daemon 能从镜像仓库拉镜像（网络 + 加速器）；③ 能创建并运行容器（存储驱动、cgroup、runc 都正常）。
- 为什么用它排障：这是最小可验证用例 —— 出问题时能立刻把故障圈到“网络/权限/运行时”其中一段。
- 加分点：把这套思路讲成方法 —— “排障时我先跑最小用例，把大问题切成可验证的小段”，这是工程素养的表达。

【☆】41 • 为什么用 curl -I 而不是浏览器？

- 因为服务器上本来就没有浏览器，也不该装（P0 生产机上装图形程序是灾难）。
 - `curl -I` 发的是 **HEAD 请求**：只取响应头，快、省流量，却能拿到最关键的信息 —— 状态码、`Server`、`Content-Type`、`Location`。
 - 可脚本化：`curl -o /dev/null -s -w '%{http_code}'` 能直接做健康检查；要握手细节用 `curl -v`。
 - 加分点：双系统环境下，“在 Windows 浏览器里看 Ubuntu 的服务”这个做法本身就**行不通**（两个系统不能同时开），所以从第一天就养成 `curl` 的习惯，等于提前进入生产工作方式。
-